NIYONKURU Jean De La Croix
223003235

Mobile Design and Development

Question 1: Which dependencies did you use and why?

I used the http package (version 1.2.1). This is the official dart package for making HTTP requests (GET, POST, PUT, DELETE). I chose it because it is light weight, easy to use, and sufficient for communicating with a REST API like JSONPlaceholder. It provides simple methods like http.get(), http.post(), http.put(), and http.delete() which basically return a Response object containing the status code and body. I also used used dart:convert which is a built-in for encoding and decoding JSON data with json.encode() and json.decode().

Question 2: How can you handle API exceptions?

I handle API exceptions using try-catch blocks in every API method. Inside the try block, I check the HTTP status code - if it's not the expected code (200 for GET/PUT/DELETE, 201 for POST), I throw a custom exception with a descriptive message. The catch block catches both network errors (no internet, timeout, DNS failure) and invalid data errors, wrapping them in a user-friendly exception message. ~~On the UI side, the user friendly exception message.~~

On the UI side, the FutureBuilder detects errors via snapshot.hasError and displays an error message with a retry button instead of crashing the app.

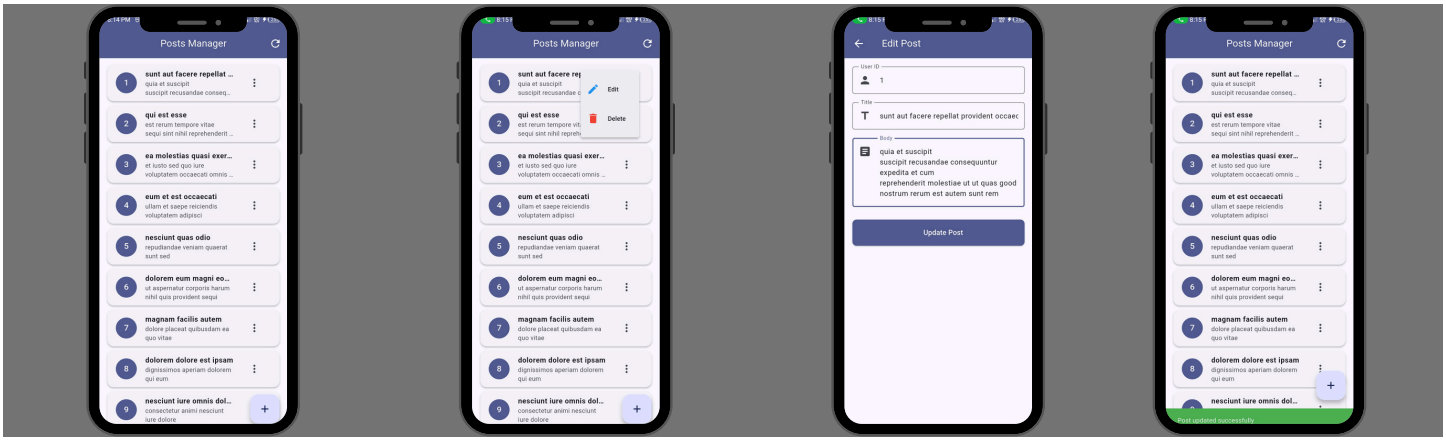
Question 3: Explain the Future Widget you used and what part of UI depends on it.

I used FutureBuilder <List<Post>> on the home screen and FutureBuilder <Post> on the detail screen. A FutureBuilder takes a Future and rebuilds the widget tree based on 3 Connection states:

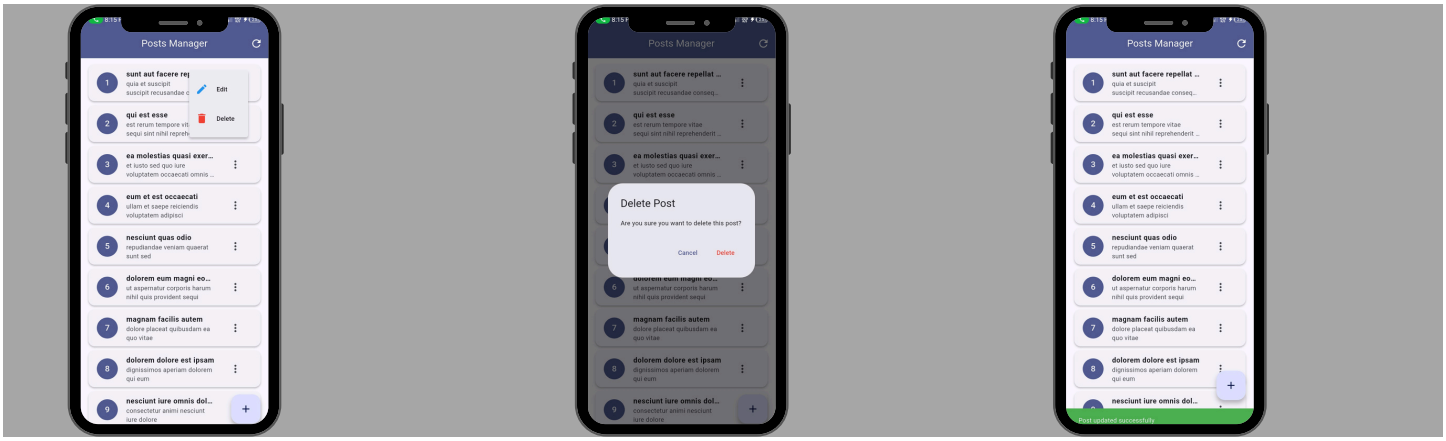
- ConnectionState.waiting → shows a CircularProgressIndicator (loading spinner)
- snapshot.hasError → shows an error icon, the error message, and a retry button.
- snapshot.hasData → shows the actual data (list of posts or post details)

The entire body of the scaffold depends on the Future. While the API call is in progress, users see a loader. Once data arrives, the ListView or detail content replaces it. This makes the UI reactive to asynchronous API calls.

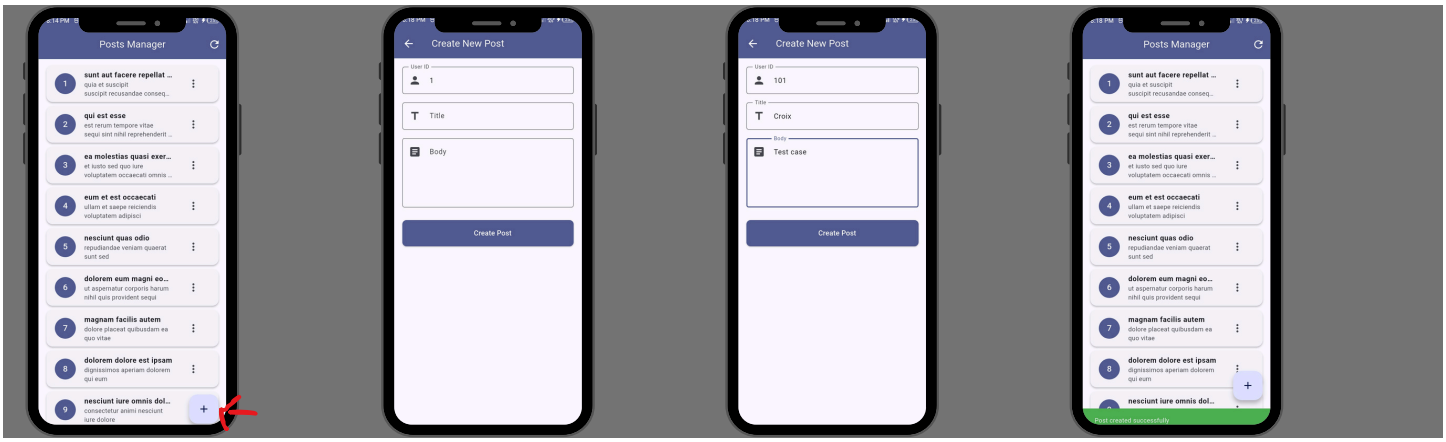
The above screens show landing page with cards and the screen on the right shows more details of the post



The above screens show how to basically edit a post



The above screens show how to delete a post



The above screens show how to create a post

They all combine together to demonstrate basic CRUD operations.

Watch a demo here: 39 MB file on MEGA